

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to $[0,1]$.
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

8. Mandatory Plots

1. Sample Images

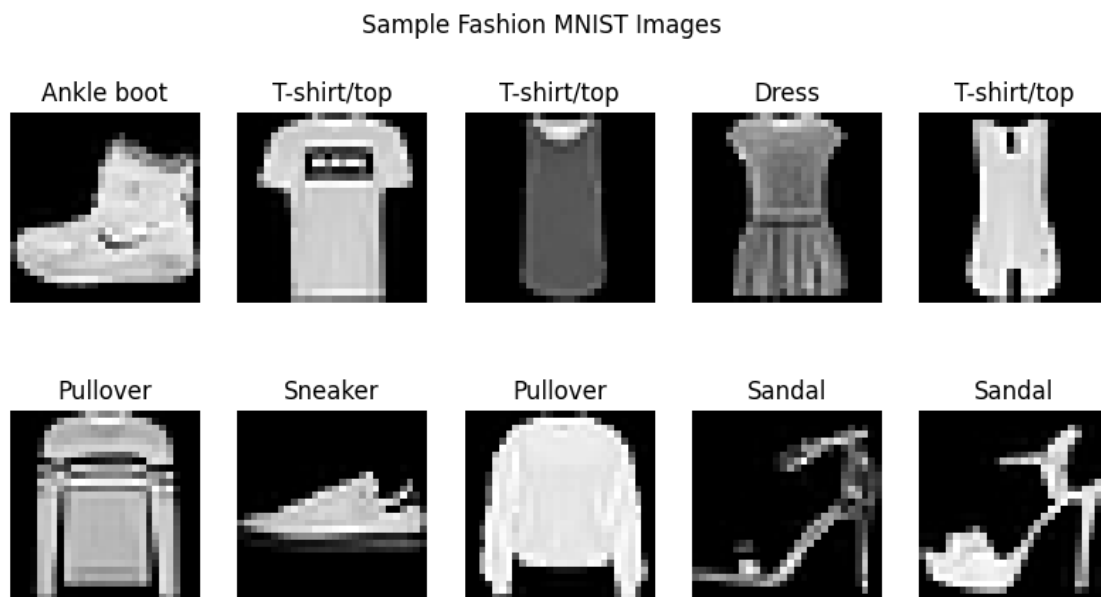


Figure 1: Sample Images from the Fashion-MNIST Dataset

Inference:

The images shown here are sample images from the MNIST(Fashion) data set. Each image is a grayscale image of size 28 into 28 pixels.

2. Class Distribution

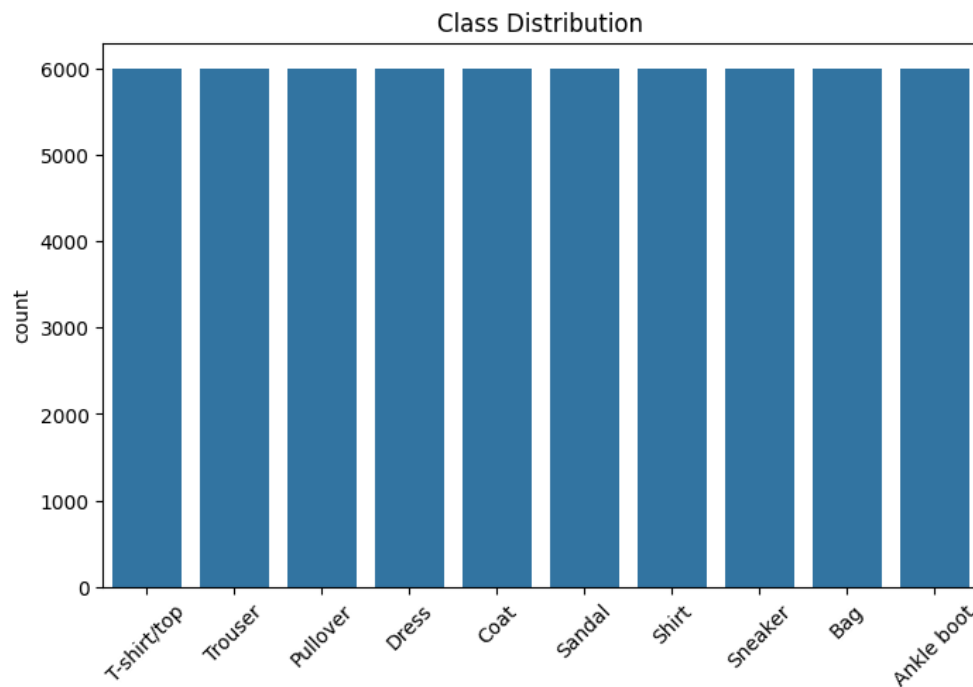


Figure 2: Class Distribution of Fashion-MNIST Dataset

Inference:

From the class distribution plot, we can see that there is approximately the same number of samples for each category. This shows the data set is balanced. This helps in attaining an accurate classification of the samples.

3. Training Accuracy vs Epoch

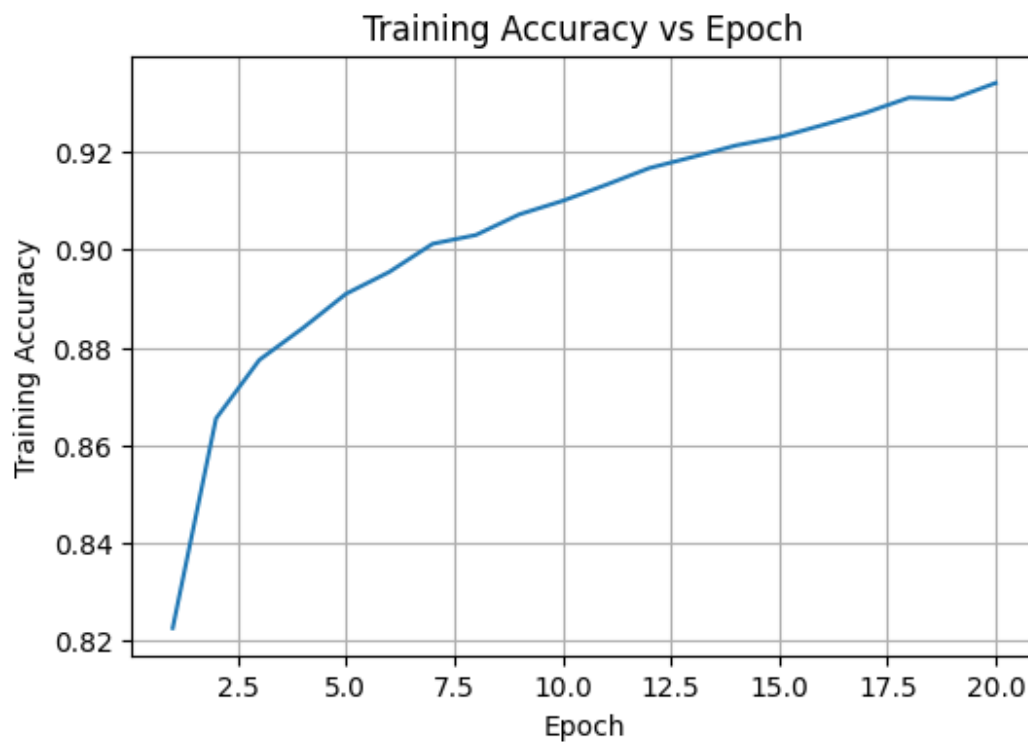


Figure 3: Training Accuracy vs Epoch

Inference:

As we can see from the plot, the training accuracy increases as number of epochs increases as it learns many useful features from the data set. But, if the model is trained for too long, it may learn the features completely and cannot perform on new data.

4. Validation Accuracy vs Epoch

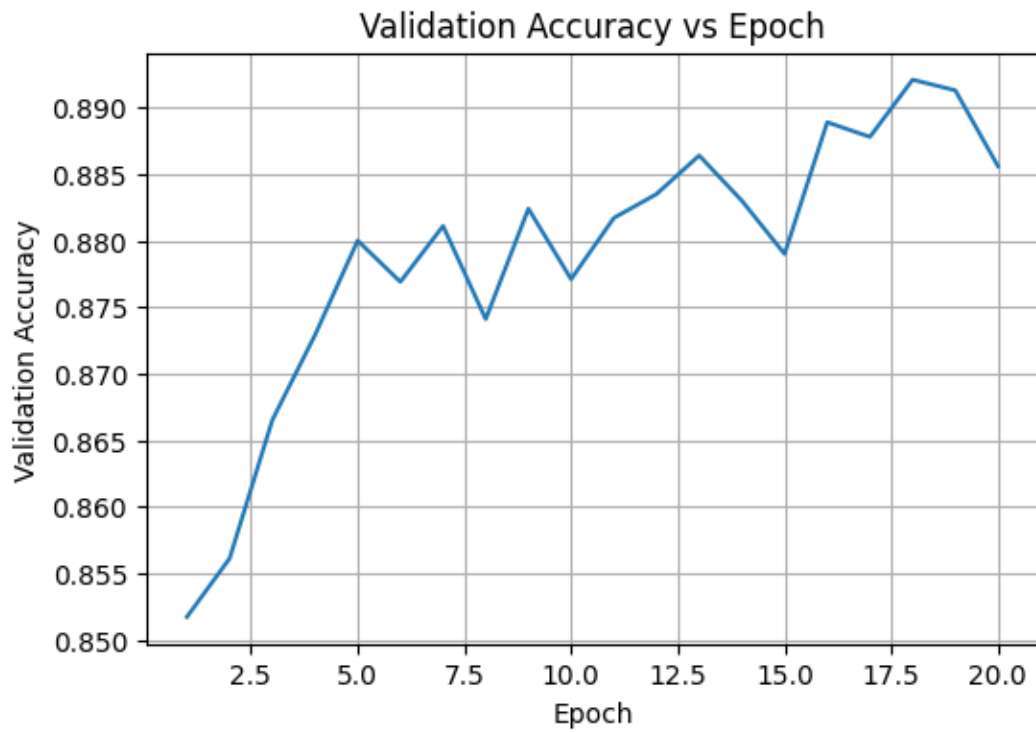


Figure 4: Validation Accuracy vs Epoch

Inference:

Validation accuracy is increasing with the number of epochs. There is very little difference between validation accuracy and training accuracy plots. So, we can say that the model performs well on new data.

5. Training Loss vs Epoch

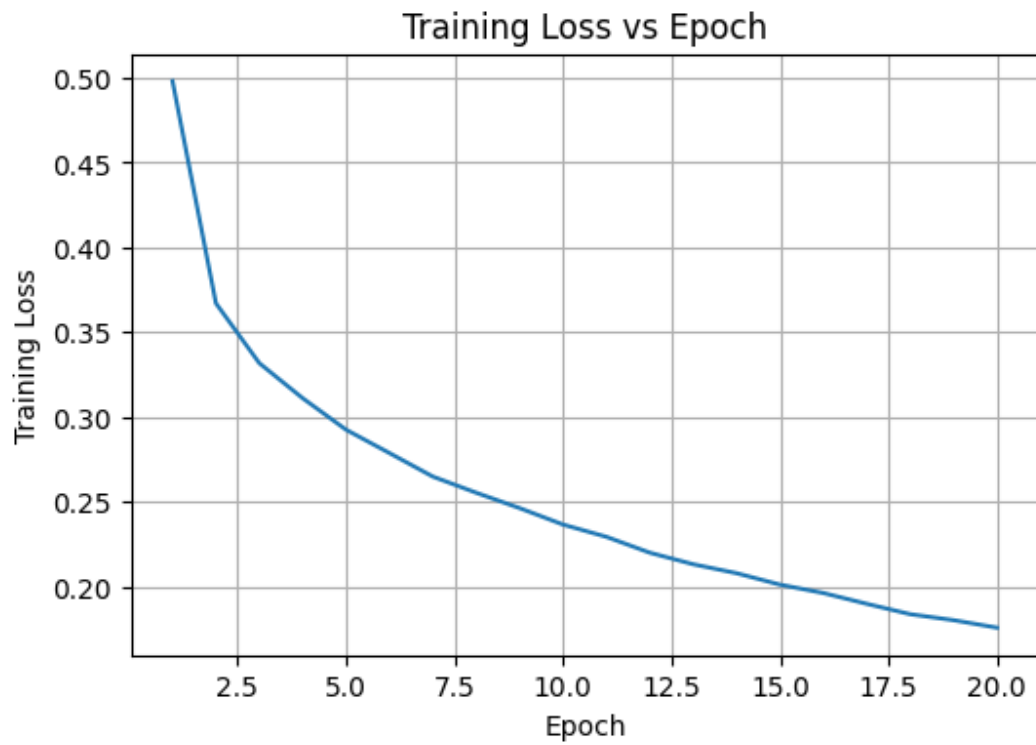


Figure 5: Training Loss vs Epoch

Inference:

Training loss is decreasing continuously as the model learns new features from the dataset. This shows that the optimizer successfully minimizes classification errors and leads to convergence.

6. Validation Loss vs Epoch

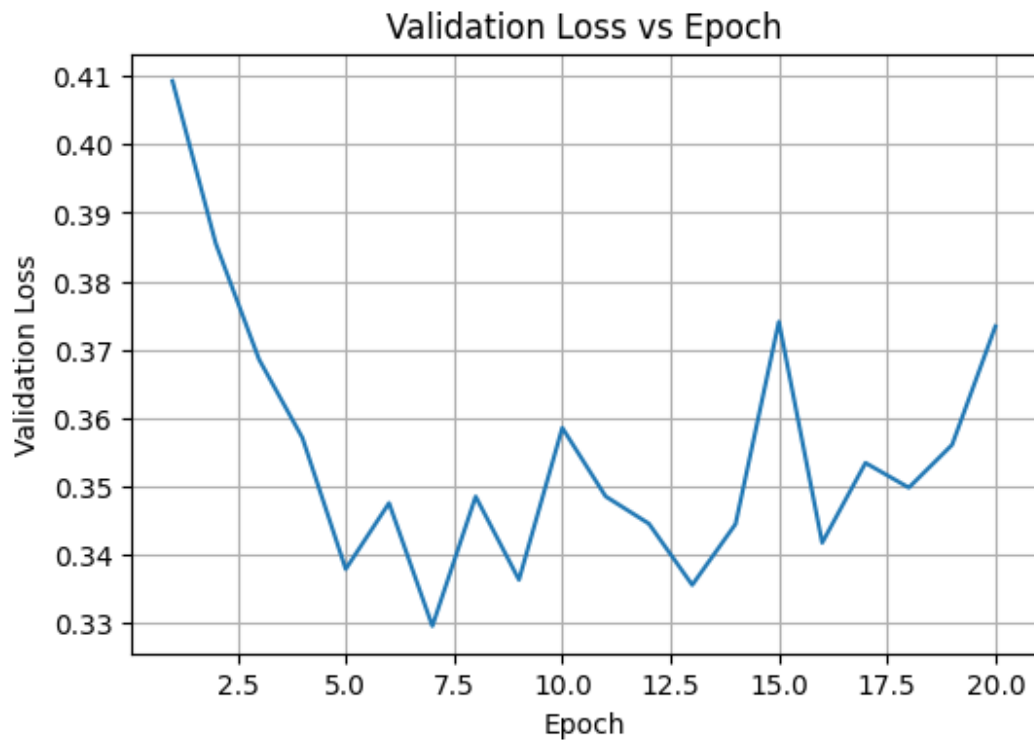


Figure 6: Validation Loss vs Epoch

Inference:

From the plot, we can see that validation loss decreases till epoch 7 approximately and then fluctuates with decreasing training loss. This indicates a little bit of overfitting.

7. Confusion Matrix

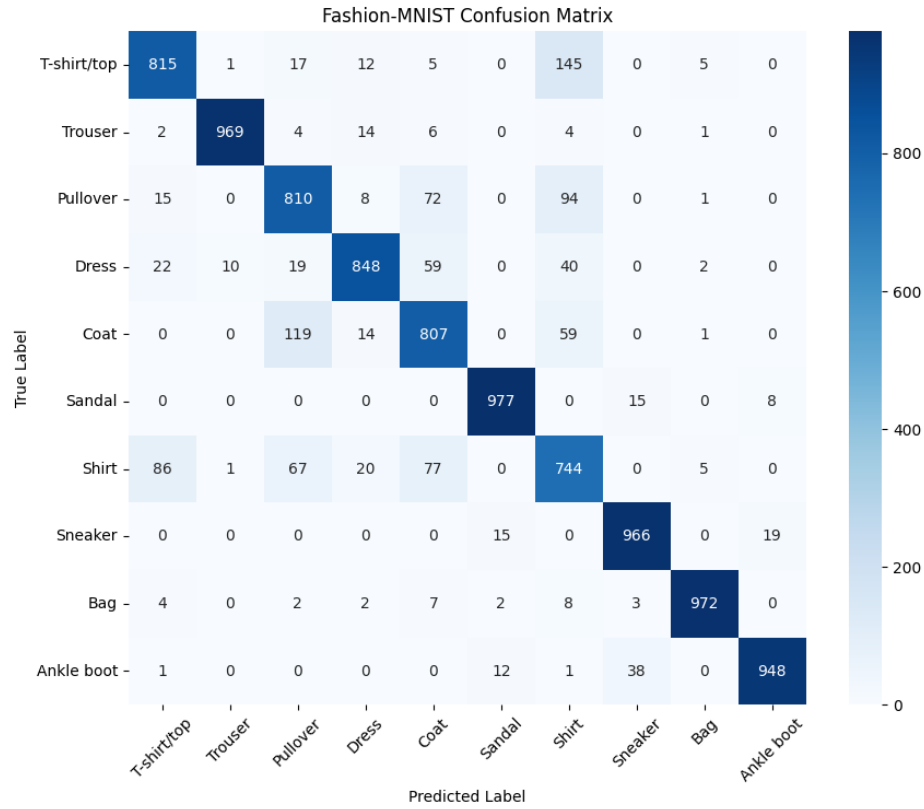


Figure 7: Confusion Matrix

Inference:

Most of the predictions are along the diagonal which means that the samples are mostly classified correctly. We can also see that similar-looking products have slightly more number of predictions (T-shirt and shirt, pullover and coat). This shows a high classification accuracy.

8. Hyperparameter Search Results

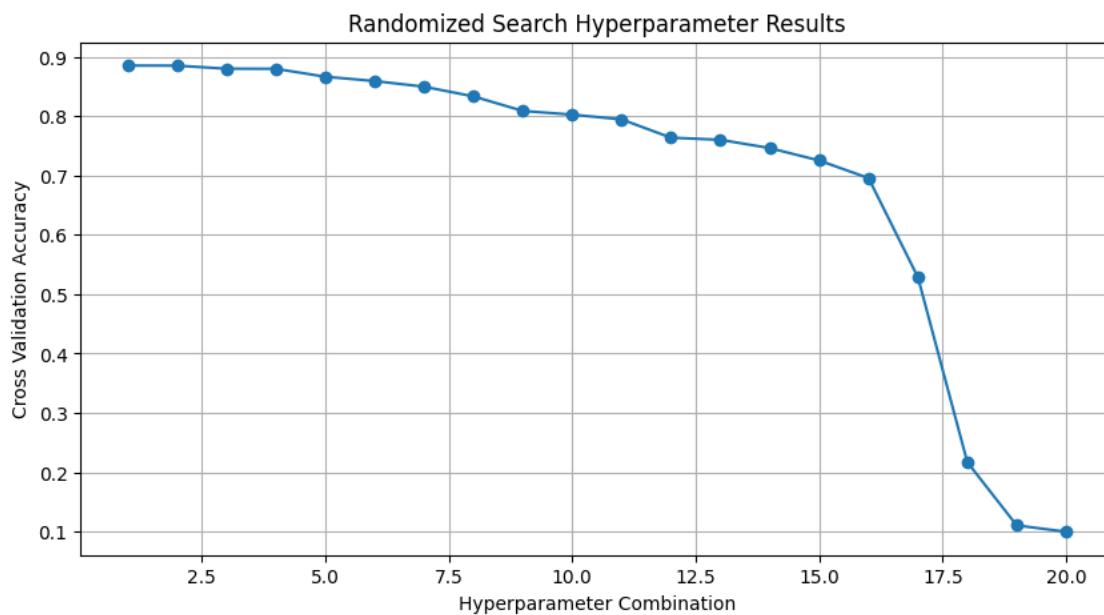


Figure 8: Hyperparameter Search Results

Inference:

The plot shows the performance of different hyperparameter combinations tested during optimization. The best combination is selected based on the highest validation accuracy. `RandomizedSearchCV` helps identify better model settings automatically.

9. Best Model Accuracy Comparison

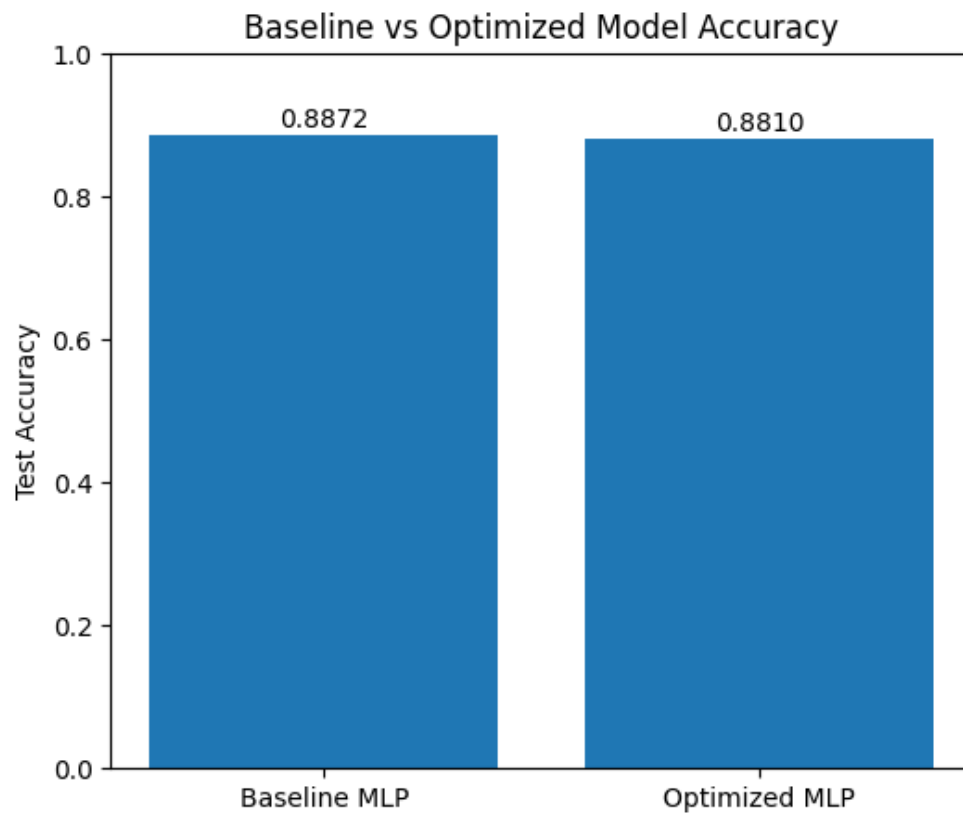


Figure 9: Baseline vs Optimized Model Accuracy

Inference:

Here, the optimized MLP accuracy is comparable to base line model for MNIST(Fashion) data set. It says that the original model classifies more accurately than hyperparameter optimized model.

Inference:

9. Results

Best Hyperparameters

Hidden Layers	1
Hidden Neurons	128
Learning Rate	0.001
Batch Size	32
Optimizer	RMSProp
Activation Function	ReLU
Epochs	20
Dropout	0.5
Cross-validation Accuracy	0.88497
Testing Accuracy	0.8810

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8872	0.8810
Precision	0.8863	0.8841
Recall	0.8872	0.8810
F1-score	0.8864	0.8812
Training Time	45.2 sec	52.6 sec

10. Discussion

Answer the following questions.

1. **Which optimization method was used?**

The optimization method used was RandomizedSearchCV .

2. **Which hyperparameters were selected?**

The best hyperparameter configuration obtained from RandomizedSearchCV is:

- Hidden Layers: 1
- Hidden Neurons: 128
- Learning Rate: 0.001
- Batch Size: 32
- Optimizer: RMSProp
- Activation Function: ReLU
- Epochs: 20
- Dropout Rate: 0.5

This configuration achieved a cross-validation accuracy of approximately 88.5%.

3. **Did optimization improve performance? Explain.**

No, the optimization did not improve the final test accuracy compared to the baseline model. The baseline MLP achieved a test accuracy of 88.72%, whereas the optimized model achieved 88.10%.

Although the optimized model performed well during cross-validation, the selected parameters did not provide better generalization on unseen test data. This shows that hyperparameter optimization does not always guarantee improved performance.

4. Which hyperparameter had the greatest impact on model performance?

The hyperparameters that had the greatest impact on model performance were the number of neurons, learning rate, and optimizer selection as we can see from Randomized CV plot.

5. Compare Grid Search and Randomized Search.

Grid search compares all possible combinations of hyper parameters which takes a lot of time. But, it provides accurate solution but not optimal. It does complete search but computationally expensive.

Randomized Search evaluates only a selected number of random combinations. It reduces training time but still manages to find best combinations of hyperparameters . It reduces computational cost. So , this is used for this experiment.

6. Recommend the best MLP configuration based on your experimental results.

Based on the experimental results, the baseline MLP configuration is recommended because it achieved the highest test accuracy of 88.72%.

The recommended architecture is:

- Input Layer: 784 neurons
- Hidden Layer 1: Dense(128, ReLU)
- Hidden Layer 2: Dense(64, ReLU)
- Output Layer: Dense(10, Softmax)
- Optimizer: Adam
- Loss Function: Categorical Cross Entropy
- Batch Size: 32
- Epochs: 20

Although the optimized model produced a comparable result, the baseline model provided slightly better test performance and is therefore recommended for Fashion-MNIST classification.

1 XOR Gate Using Perceptron

1.1 Weight Updates During Training

Table 1: Weight and Bias Updates for XOR Gate

Update	Weight 1	Weight 2	Bias
1	0	0	-1
2	0	1	0
3	-1	0	-1
4	-1	1	0
5	0	1	1
6	-1	0	0
7	-1	0	-1
8	-1	1	0
9	0	1	1
10	-1	0	0

1.2 Decision Boundary After Each Weight Update

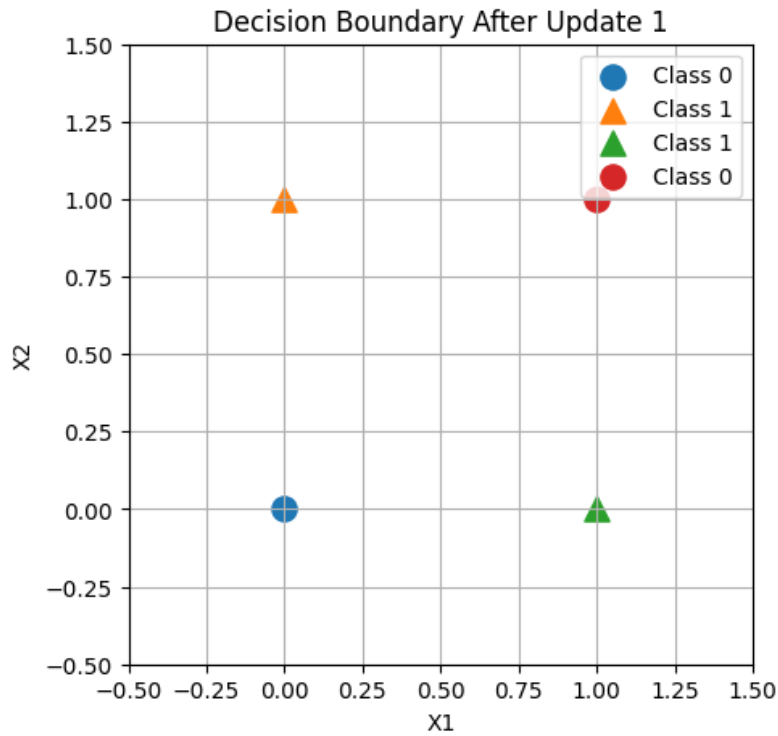


Figure 10: Decision Boundary After Weight Update 1

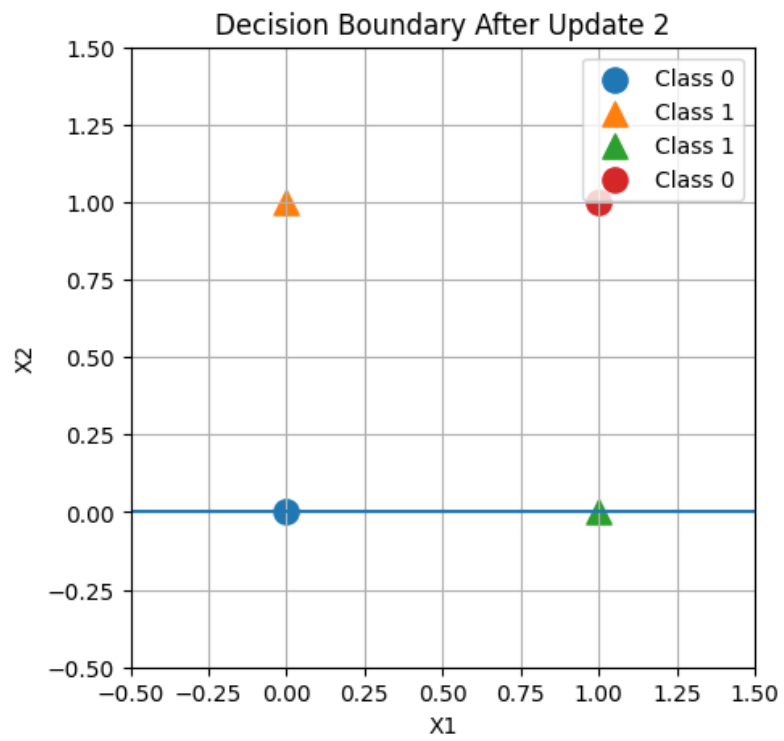


Figure 11: Decision Boundary After Weight Update 2

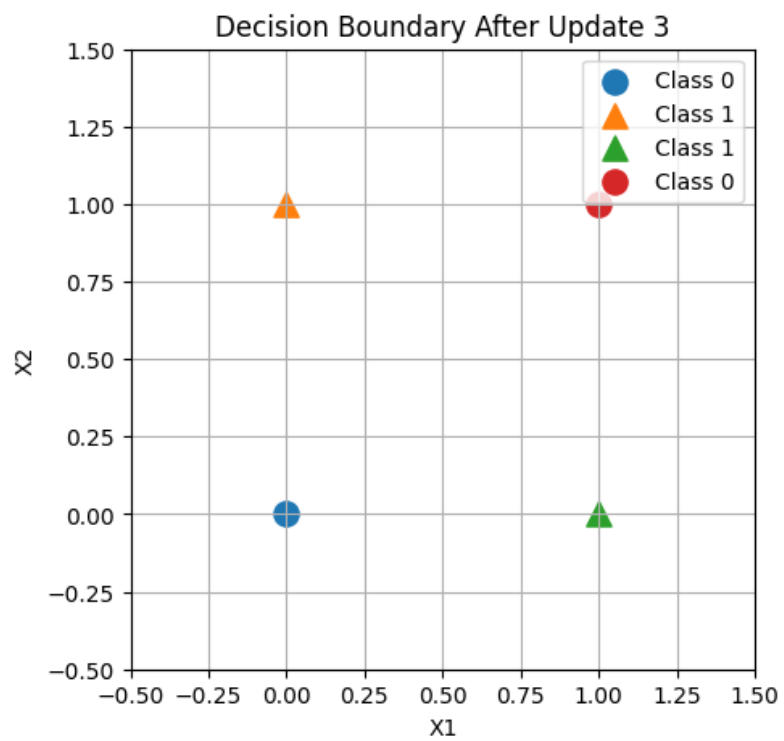


Figure 12: Decision Boundary After Weight Update 3

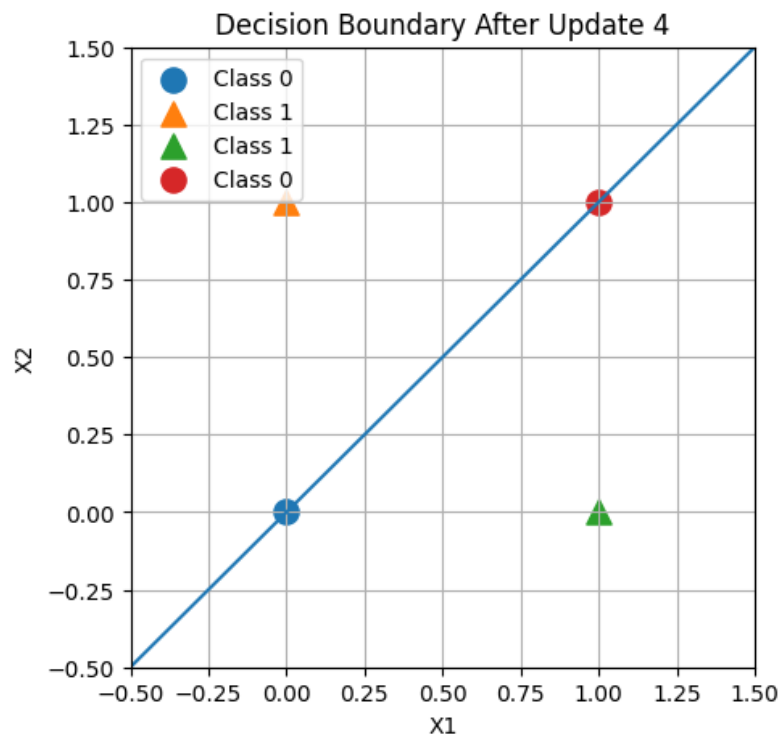


Figure 13: Decision Boundary After Weight Update 4

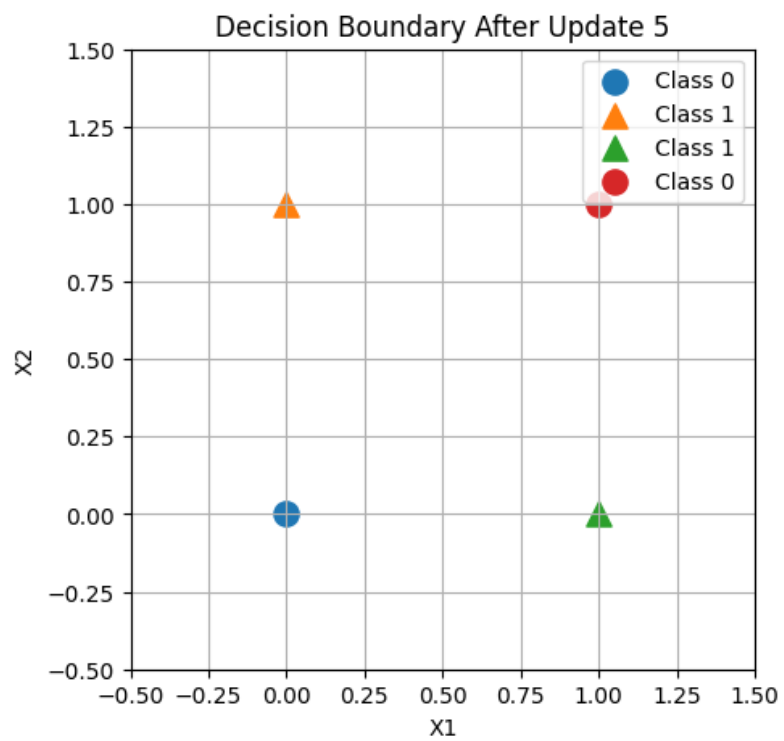


Figure 14: Decision Boundary After Weight Update 5

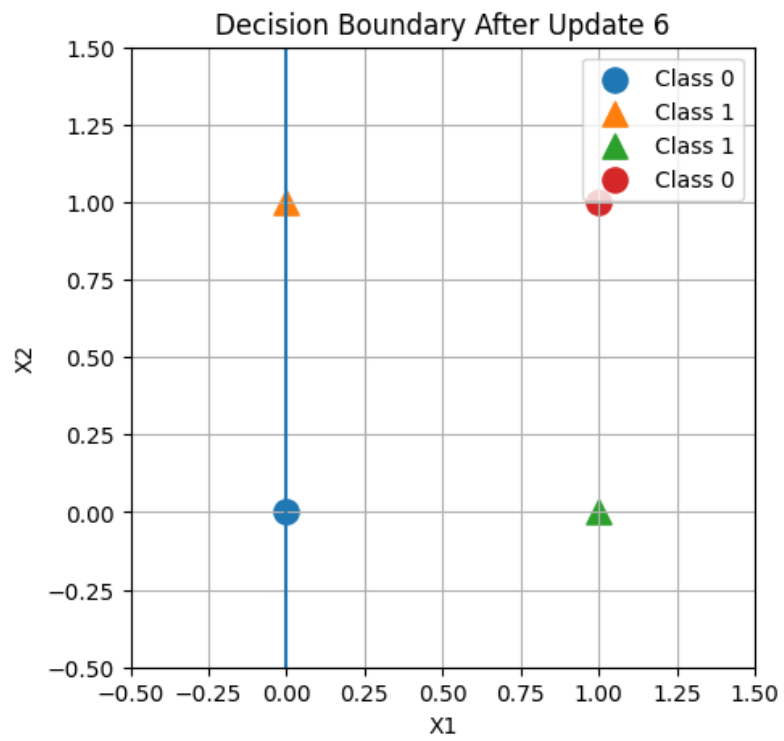


Figure 15: Decision Boundary After Weight Update 6

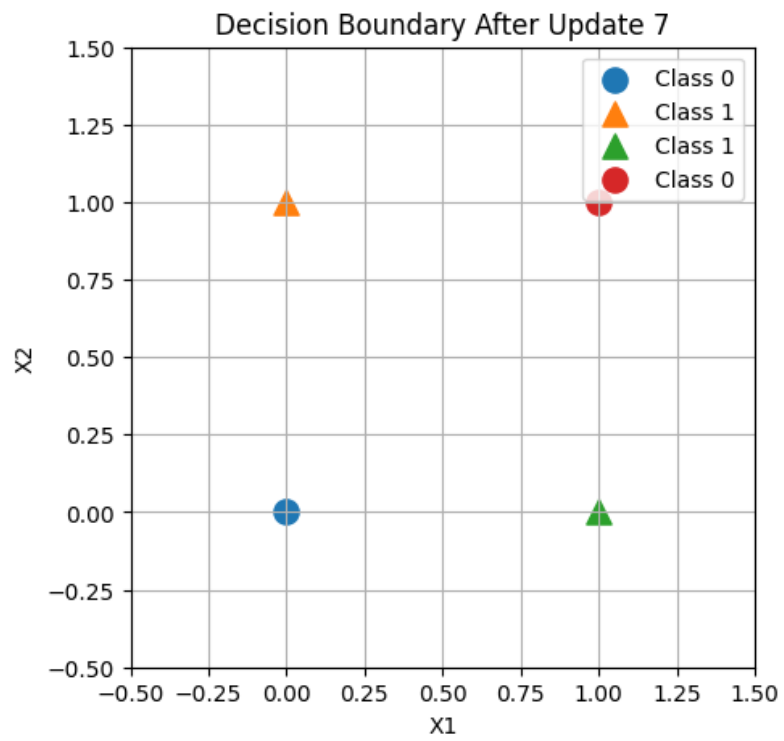


Figure 16: Decision Boundary After Weight Update 7

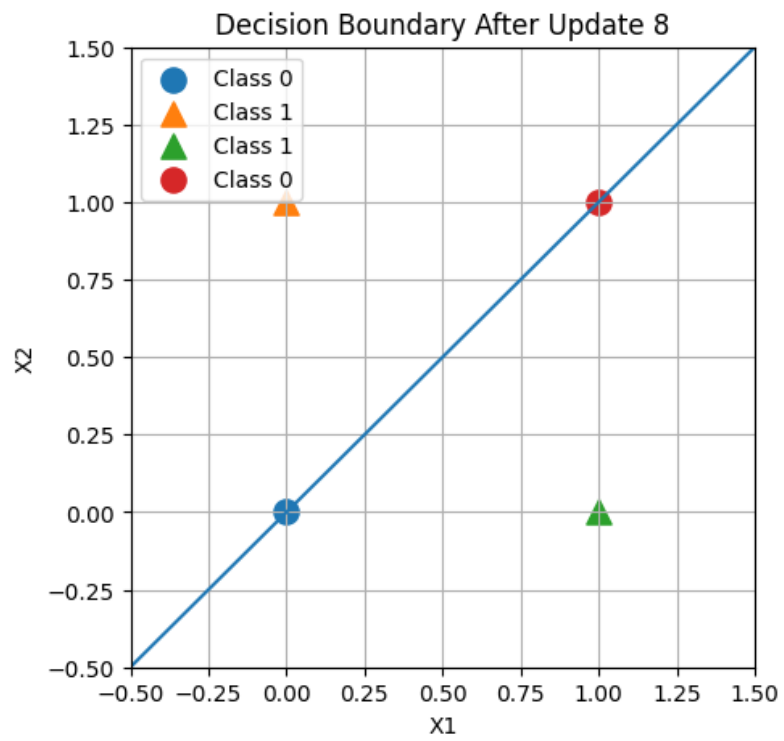


Figure 17: Decision Boundary After Weight Update 8

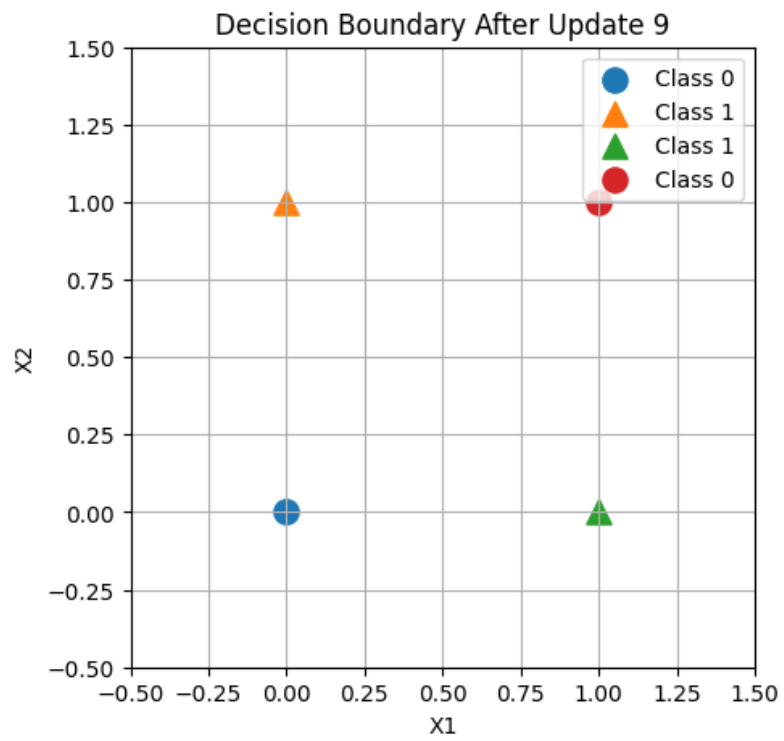


Figure 18: Decision Boundary After Weight Update 9



Figure 19: Decision Boundary After Weight Update 10

1.3 Inference

As we can see from the graph, the points belonging to one class are on the diagonal. It cannot be classified by a linear decision boundary. Single perceptron can only classify linear data. So, we do not reach convergence.

From the weight update table, the change in the weights does not follow one direction. It oscillates frequently which says that it does not achieve convergence.

11. Report Contents

Include Objective, Theory, Dataset, Experimental Procedure, Source Code, Hyperparameter Optimization, Results, Plots with Inference, Discussion, Conclusion and References

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.

GitHub Repository

[https://github.com/Nitheesha-1025/Deep-Learning/tree/Perceptron-Algorithm-\(Experiment-2\)](https://github.com/Nitheesha-1025/Deep-Learning/tree/Perceptron-Algorithm-(Experiment-2))